



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Plataforma web para la
descarga centralizada de
software con gestión de
bundles y búsqueda semántica
mediante LLMs
Documentación técnica**



Presentado por José Gallardo Caballero
en Universidad de Burgos — 23 de junio
de 2026

Tutores: José Manuel Aroca Fernández,
Rodrigo Pascual García

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Lista de siglas, acrónimos y abreviaturas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Metodología	1
A.3. Planificación temporal	2
A.4. Hitos	3
A.5. Gestión de riesgos	3
A.6. Estudio de viabilidad	4
A.7. Criterios de seguimiento	6
Apéndice B Especificación de Requisitos	7
B.1. Introducción	7
B.2. Actores	7
B.3. Objetivos generales	7
B.4. Catálogo de requisitos funcionales	8
B.5. Requisitos no funcionales	10
B.6. Reglas de negocio	11
B.7. Especificación de casos de uso	12
B.8. Trazabilidad inicial	16

Apéndice C Especificación de diseño	17
C.1. Introducción	17
C.2. Diseño de datos	17
C.3. Diseño arquitectónico	20
C.4. Diseño procedimental	22
C.5. Diseño de seguridad	24
C.6. Diseño de despliegue	25
Apéndice D TODO: Documentación técnica de programación	26
D.1. Introducción	26
D.2. Estructura de directorios	26
D.3. Configuración del entorno	26
D.4. Manual del programador	27
D.5. Compilación, instalación y ejecución del proyecto	27
D.6. Pruebas del sistema	27
D.7. Operación y resolución de incidencias	27
Apéndice E TODO: Documentación de usuario	28
E.1. Introducción	28
E.2. Requisitos de usuario	28
E.3. Acceso e instalación	28
E.4. Uso del catálogo	28
E.5. Bundles y estrellas	29
E.6. Búsqueda semántica	29
E.7. Administración	29
E.8. Errores frecuentes y soporte	29
Apéndice F Anexo de sostenibilización curricular	30
F.1. Introducción	30
F.2. SOS1: contextualización crítica	30
F.3. SOS2: uso sostenible de recursos	31
F.4. SOS3: participación	31
F.5. SOS4: principios éticos	32
F.6. Conclusión personal	32
Bibliografía	33

Índice de figuras

Índice de tablas

A.1. Planificación provisional por fases.	2
A.2. Riesgos principales y mitigaciones.	3
A.3. Estimación económica orientativa.	5
B.1. Requisitos funcionales.	8
B.2. Requisitos no funcionales.	10
B.3. CU-01 Generar ZIP.	12
B.4. CU-02 Crear y publicar bundle.	13
B.5. CU-03 Personalizar un bundle público.	13
B.6. CU-04 Asignar estrella.	14
B.7. CU-05 Mantener una fuente.	14
B.8. CU-05b Solicitar una aplicación.	15
B.9. CU-06 Buscar mediante lenguaje natural.	15
B.10. Trazabilidad entre áreas y pruebas.	16
C.1. Entidades transaccionales principales.	17
C.2. Componentes lógicos y criterio de escalado.	21
C.3. Operaciones principales de la API.	21

Lista de siglas, acrónimos y abreviaturas

Esta lista reúne las abreviaturas utilizadas en la memoria y en la documentación técnica, junto con su significado.

Sigla	Significado
AMQP	<i>Advanced Message Queuing Protocol</i> (protocolo avanzado de colas de mensajes)
API	<i>Application Programming Interface</i> (interfaz de programación de aplicaciones)
CAPTCHA	<i>Completely Automated Public Turing test to tell Computers and Humans Apart</i>
CDN	<i>Content Delivery Network</i> (red de distribución de contenidos)
CRUE	CRUE Universidades Españolas; originalmente, Conferencia de Rectores de las Universidades Españolas
CSS	<i>Cascading Style Sheets</i> (hojas de estilo en cascada)
CU	Caso de uso
DNS	<i>Domain Name System</i> (sistema de nombres de dominio)
DTO	<i>Data Transfer Object</i> (objeto de transferencia de datos)
E2E	<i>End to End</i> (extremo a extremo)
ES	<i>ECMAScript</i>
HTML	<i>HyperText Markup Language</i> (lenguaje de marcado de hipertexto)
HTTPS	<i>HyperText Transfer Protocol Secure</i>
IA	<i>Inteligencia artificial</i>
ID	<i>Identifier</i> (identificador)

Continúa en la página siguiente

Lista de siglas, acrónimos y abreviaturas (continuación)	
Sigla	Significado
IP	<i>Internet Protocol</i> (protocolo de Internet)
JPA	<i>Jakarta Persistence API</i>
LLM	<i>Large Language Model</i> (modelo de lenguaje de gran tamaño)
ODS	<i>Objetivos de Desarrollo Sostenible</i>
REST	<i>Representational State Transfer</i> (transferencia de estado representacional)
SOS1–SOS4	Competencias de sostenibilidad 1 a 4
SSRF	<i>Server-Side Request Forgery</i> (falsificación de peticiones del lado del servidor)
URL	<i>Uniform Resource Locator</i> (localizador uniforme de recursos)
VPS	<i>Virtual Private Server</i> (servidor privado virtual)
WCAG	<i>Web Content Accessibility Guidelines</i> (pautas de accesibilidad para el contenido web)

Apéndice A

Plan de Proyecto Software

A.1. Introducción

TODO: Este anexo establece una planificación provisional para transformar el diseño de Batch Downloader en una aplicación verificable. Las semanas son relativas al inicio de la implementación, por lo que pueden adaptarse al calendario académico sin alterar las dependencias entre fases.

El trabajo seguirá un enfoque iterativo. Cada fase terminará con una demostración, pruebas automatizadas y actualización de la documentación. Las funcionalidades de inteligencia artificial y despliegue distribuido se abordarán después de validar el núcleo de descarga.

A.2. Metodología

Se utilizará un tablero de tareas con una lista priorizada y ciclos semanales. Cada elemento deberá incluir descripción, criterios de aceptación, riesgos y pruebas. Una tarea se considerará terminada cuando:

- el código esté integrado en la rama principal del proyecto;
- las pruebas relacionadas sean reproducibles;
- no exponga secretos ni datos personales;
- la documentación afectada esté actualizada;
- el flujo pueda demostrarse en el entorno de desarrollo.

Las revisiones con los tutores se utilizarán para confirmar alcance, arquitectura y presentación de resultados. Los cambios relevantes se registrarán como decisiones de arquitectura para evitar que la memoria y la implementación evolucionen de forma contradictoria.

A.3. Planificación temporal

Tabla A.1: Planificación provisional por fases.

Semanas	Fase	Resultados esperados
1–2	Preparación	Repositorios, convenciones, entornos Docker, integración continua, modelos iniciales y contrato OpenAPI.
3–4	Catálogo y seguridad base	Usuarios, roles, aplicaciones, fuentes, etiquetas, validación de entradas y panel administrativo mínimo.
5–7	Resolución y descargas	Playwright, dominios permitidos, protección SSRF, RabbitMQ, workers y estados de trabajos.
8–9	ZIP y temporales	Descarga controlada, manifiesto, resultado parcial, MinIO temporal, cancelación y limpieza automática.
10–11	Bundles y estrellas	Bundles públicos, privados y oficiales, personalización temporal, enlaces, clonación y estrella binaria.
12	Solicitudes y avisos	Peticiones de nuevo software, revisión administrativa y notificaciones configurables.
13–14	Búsqueda semántica	Embeddings, PostgreSQL con pgvector, consultas por intención y filtros de negocio.
15	Asistencia con LLM	Recomendaciones explicables, borradores de bundles, validación de esquemas y resumen de errores.
16–17	Pruebas y rendimiento	Integración, extremo a extremo, seguridad, carga, concurrencia y consumo de almacenamiento.

Semanas	Fase	Resultados esperados
18	Despliegue y cierre	Imágenes definitivas, demostración, documentación, manuales y preparación de defensa.

A.4. Hitos

H1 - Catálogo utilizable: un administrador puede registrar aplicaciones y un usuario puede buscarlas.

H2 - Descarga segura: una selección genera un ZIP desde dominios autorizados y elimina los temporales.

H3 - Comunidad: los usuarios pueden crear, publicar, personalizar y valorar bundles.

H4 - Búsqueda semántica: una consulta por intención devuelve aplicaciones válidas del catálogo.

H5 - Entrega: el sistema se despliega, se prueba y los manuales reflejan el comportamiento real.

A.5. Gestión de riesgos

Tabla A.2: Riesgos principales y mitigaciones.

Riesgo	Prob.	Impacto	Mitigación
Cambios frecuentes en páginas oficiales	Alta	Alta	Resolver genérico, resolvers específicos, pruebas programadas y revisión manual.
Uso del servidor como proxy de descargas	Media	Alta	Cuotas, rate limiting, límites por IP y usuario, CAPTCHA opcional y auditoría.
Ataques SSRF	Media	Crítico	Allowlist, validación DNS e IP tras cada redirección, HTTPS y bloqueo de redes internas.

Riesgo	Prob.	Impacto	Mitigación
Consumo excesivo de disco o red	Media	Alta	Límites, streaming futuro, expiración, limpieza periódica y métricas.
Incompatibilidad entre Java y dependencias	Baja	Media	Prototipo inicial y posibilidad de adoptar una versión LTS compatible.
Respuestas incorrectas del LLM	Alta	Media	Recuperación cerrada, esquemas, filtros deterministas y degradación sin LLM.
Alcance excesivo	Alta	Alta	Priorizar el núcleo, resolvers avanzados y modelo local después del producto mínimo.
Falta de tiempo para manuales	Media	Media	Mantener esqueletos y completarlos durante las pruebas de aceptación.

A.6. Estudio de viabilidad

Viabilidad técnica

Las tecnologías seleccionadas disponen de documentación, bibliotecas para Java y ejecución mediante contenedores. El principal riesgo técnico no es la disponibilidad de herramientas, sino la variabilidad de las fuentes externas y el tratamiento seguro de descargas. Por este motivo se requiere un prototipo temprano con varias páginas de distinta complejidad.

La arquitectura puede simplificarse para el desarrollo local: un frontend, una aplicación Spring modular, un worker y servicios Docker.

Viabilidad económica

La siguiente estimación representa el coste equivalente del trabajo y de la infraestructura de desarrollo. No corresponde a gastos ya realizados.

Concepto	Cantidad	Precio estimado	Subtotal
Trabajo de análisis, desarrollo y documentación	480 h	15,00 €/h	7200,00 €
Infraestructura VPS	5 meses	15,00 €/mes	75,00 €
Dominio	1	15,00 €	15,00 €
Total estimado			7290,00 €

Tabla A.3: Estimación económica orientativa.

TODO: Para la fase académica podrán utilizarse herramientas de código abierto y niveles gratuitos. El proveedor LLM deberá configurarse con límites de gasto. Un despliegue público real necesitaría revisar costes de transferencia, almacenamiento temporal, observabilidad y soporte.

Viabilidad legal

La plataforma tratará cuentas, direcciones IP, registros y solicitudes, por lo que deberá cumplir el Reglamento General de Protección de Datos [5] y la Ley Orgánica 3/2018 [4]. Se aplicarán minimización, limitación de finalidad, plazos de conservación, información al usuario y mecanismos para ejercer derechos.

Si se presta como servicio web, deberán publicarse la identificación del responsable, condiciones de uso, política de privacidad y política de cookies conforme a la Ley 34/2002 [3]. Los correos solo se enviarán para finalidades solicitadas y con mecanismos para evitar abuso.

Los instaladores pertenecen a sus respectivos titulares. Batch Downloader no alterará ni redistribuirá copias permanentes; los procesará temporalmente desde fuentes oficiales para atender una solicitud. Aun así, antes de incorporar cada aplicación deberán revisarse sus condiciones de uso y las limitaciones de automatización. La Ley de Propiedad Intelectual constituye el marco general español aplicable [2].

TODO: El código propio se publicará bajo una licencia permisiva, previsiblemente GPL 3.0, tras confirmar que todas las dependencias y recursos son compatibles. La memoria conservará su licencia documental actual. Los nombres y logotipos de terceros se utilizarán únicamente con finalidad identificativa y deberán respetar sus marcas.

A.7. Criterios de seguimiento

El avance se medirá por hitos demostrables y cobertura de riesgos, no únicamente por número de funcionalidades. En cada revisión se registrarán tareas completadas, incidencias abiertas, pruebas fallidas, decisiones pendientes y desviación respecto a la planificación. Si el calendario se reduce, se protegerán primero la seguridad, la descarga temporal y la trazabilidad; las capacidades semánticas y el despliegue avanzado se pospondrán antes que degradar esas garantías.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este documento define el comportamiento esperado de Batch Downloader antes de su implementación. Los requisitos se identifican para permitir trazabilidad con casos de uso, diseño y pruebas.

B.2. Actores

Usuario anónimo: consulta el catálogo, crea selecciones temporales y descarga bundles públicos dentro de cuotas.

Usuario registrado: conserva bundles, los publica, asigna estrellas, solicita software y consulta sus trabajos.

Administrador: mantiene catálogo, fuentes, dominios, resolvers y estados; revisa solicitudes y registros.

Sistema externo oficial: página o servicio del editor desde el que se resuelve y descarga un instalador.

Proveedor semántico: servicio de embeddings o LLM utilizado detrás de una interfaz controlada.

B.3. Objetivos generales

- Reducir el trabajo necesario para descargar varias aplicaciones.

- Mantener el origen oficial y la trazabilidad de cada instalador.
- Evitar almacenamiento permanente y eliminar artefactos temporales.
- Permitir reutilizar selecciones mediante bundles.
- Mejorar el descubrimiento mediante búsqueda semántica segura.
- Proteger la infraestructura frente a URL maliciosas y uso abusivo.

B.4. Catálogo de requisitos funcionales

Tabla B.1: Requisitos funcionales.

ID	Descripción	Prioridad	Actor
RF-01	Mostrar las aplicaciones activas con nombre, descripción, sistemas, arquitecturas, etiquetas y estado.	Alta	Todos
RF-02	Buscar aplicaciones por coincidencia parcial de nombre.	Alta	Todos
RF-03	Filtrar por sistema operativo, arquitectura y etiquetas, exigiendo todas o un mínimo de etiquetas.	Alta	Todos
RF-04	Añadir o retirar aplicaciones de una selección temporal.	Alta	Todos
RF-05	Mostrar tamaño estimado, fuentes y advertencias antes de generar el ZIP.	Media	Todos
RF-06	Crear un trabajo asíncrono para generar el ZIP y devolver su identificador.	Alta	Todos
RF-07	Consultar el progreso global y el estado individual de cada aplicación.	Alta	Todos
RF-08	Permitir cancelar un trabajo que todavía no haya finalizado.	Media	Todos
RF-09	Descargar un ZIP listo antes de su expiración.	Alta	Todos
RF-10	Producir un ZIP parcial cuando alguna aplicación falle y exista al menos una descarga válida.	Alta	Sistema
RF-11	Incluir manifiesto, documento informativo y errores individuales en el ZIP.	Alta	Sistema

ID	Descripción	Prioridad	Actor
RF-12	Registrar y eliminar los archivos temporales al finalizar, cancelar o expirar.	Alta	Sistema
RF-13	Registrar, iniciar y cerrar sesión con controles de seguridad.	Alta	Registrado
RF-14	Crear, editar y eliminar bundles propios.	Alta	Registrado
RF-15	Definir un bundle propio como privado o público y generar un enlace compartible si es público.	Alta	Registrado
RF-16	Personalizar temporalmente cualquier bundle descargable sin modificar el original.	Alta	Todos
RF-17	Clonar un bundle público como nuevo bundle del usuario.	Media	Registrado
RF-18	Asignar o retirar una estrella de un bundle público ajeno, con un máximo de una por usuario.	Media	Registrado
RF-19	Ordenar bundles por estrellas, fecha, nombre o etiquetas.	Media	Todos
RF-20	Solicitar la incorporación de software indicando nombre, descripción y página oficial.	Media	Registrado
RF-21	Añadir, editar, desactivar y reactivar aplicaciones, fuentes y etiquetas.	Alta	Administrador
RF-22	Configurar dominios permitidos y estrategia de resolución por fuente.	Alta	Administrador
RF-23	Probar un resolver y consultar sus redirecciones, metadatos y errores.	Alta	Administrador
RF-24	Revisar trabajos, fallos de resolución, solicitudes y acciones administrativas.	Alta	Administrador
RF-25	Ejecutar comprobaciones periódicas y marcar fuentes para revisión manual.	Media	Sistema
RF-26	Buscar aplicaciones mediante lenguaje natural y devolver candidatos del catálogo.	Media	Todos
RF-27	Proponer un bundle editable a partir de una consulta semántica.	Media	Todos
RF-28	Explicar recomendaciones y resumir errores sin ejecutar acciones críticas.	Media	Todos
RF-29	Enviar un aviso de finalización por correo cuando el usuario lo solicite y exista una dirección verificada.	Baja	Registrado

ID	Descripción	Prioridad	Actor
RF-30	Permitir a un administrador crear y mantener bundles oficiales.	Media	Administrador

B.5. Requisitos no funcionales

Tabla B.2: Requisitos no funcionales.

ID	Descripción verificable	Categoría
RNF-01	Solo se aceptarán descargas HTTPS desde dominios permitidos; se bloquearán redes privadas, locales y reservadas antes y después de redirecciones.	Seguridad
RNF-02	Ningún instalador se almacenará permanentemente. Los temporales y ZIP expirados se eliminarán mediante políticas automáticas verificables.	Privacidad
RNF-03	Los nombres incorporados al ZIP no podrán contener rutas, secuencias de escape, nombres reservados ni colisiones sin resolver.	Seguridad
RNF-04	Se aplicarán límites configurables de tamaño, aplicaciones, tiempo, redirecciones, reintentos y concurrencia.	Seguridad
RNF-05	Los permisos se comprobarán en el backend para cada operación protegida.	Seguridad
RNF-06	Las contraseñas se almacenarán con un algoritmo de derivación resistente y nunca aparecerán en registros.	Seguridad
RNF-07	Las operaciones administrativas y el uso de claves de API quedarán auditados.	Trazabilidad
RNF-08	La API se documentará mediante OpenAPI y mantendrá errores estructurados.	Mantenibilidad
RNF-09	La creación de un trabajo responderá sin esperar a que terminen las descargas.	Rendimiento
RNF-10	La interfaz mostrará progreso y errores con lenguaje comprensible y será operable con teclado.	Usabilidad
RNF-11	Los componentes se ejecutarán en contenedores y podrán configurarse mediante variables y secretos externos.	Portabilidad

ID	Descripción verificable	Categoría
RNF-12	La API y los workers podrán replicarse de forma independiente.	Escalabilidad
RNF-13	El sistema dispondrá de métricas de trabajos, colas, errores, temporales y tiempos por fase.	Observabilidad
RNF-14	El LLM solo recibirá el contexto mínimo, su salida se validará y el sistema continuará sin él cuando falle.	Fiabilidad
RNF-15	Las pruebas automatizadas cubrirán reglas críticas de URL, dominios, permisos, ZIP y estados.	Calidad
RNF-16	Los datos personales se conservarán durante plazos definidos y podrán ser exportados o eliminados conforme a la normativa.	Legal
RNF-17	Las páginas principales deberán adaptarse a escritorio y dispositivos móviles sin ocultar información esencial.	Usabilidad
RNF-18	La interfaz distinguirá claramente página oficial, dominio final, tamaño conocido o estimado y fecha de comprobación.	Transparencia

B.6. Reglas de negocio

1. Una selección de dos o más aplicaciones se tratará como bundle temporal, aunque no se guarde.
2. Un usuario solo podrá mantener una estrella activa por bundle.
3. La personalización de un bundle ajeno no alterará el original.
4. Solo el propietario podrá editar o eliminar permanentemente su bundle; un administrador podrá moderarlo según las condiciones de uso.
5. Un bundle oficial solo podrá ser creado o marcado como tal por un administrador.
6. Un bundle privado no será accesible mediante su antiguo enlace público.
7. La descripción de una aplicación o de un bundle será un texto redactado manualmente por un usuario; el sistema no la generará ni la completará automáticamente.

8. La fuente de verdad del catálogo será MySQL; pgvector mantendrá una representación derivada.
9. Una sugerencia del LLM nunca evitará la validación normal de aplicaciones y descargas.
10. El trabajo parcial será válido cuando incluya al menos un instalador y describa todos los fallos.
11. Un trabajo listo dejará de estar disponible cuando se descargue, se elimine o venza su plazo.

B.7. Especificación de casos de uso

CU-01 Generar ZIP

Actor	Usuario anónimo o registrado
Requisitos	RF-01 a RF-12; RNF-01 a RNF-05
Precondición	Existe al menos una aplicación seleccionada y no se superan las cuotas iniciales.
Flujo principal	1. El usuario revisa selección, sistema y arquitectura. 2. El sistema estima tamaño y muestra fuentes. 3. El usuario confirma. 4. La API valida y crea el trabajo. 5. El worker resuelve y descarga cada aplicación. 6. El sistema genera el ZIP y notifica que está listo. 7. El usuario descarga el archivo.
Alternativas	Una aplicación puede fallar y producirse un ZIP parcial. El usuario puede cancelar. Si todas fallan, el trabajo termina como fallido.
Postcondición	El resultado queda disponible temporalmente y los artefactos se eliminan al expirar.

Tabla B.3: CU-01 Generar ZIP.

CU-02 Crear y publicar bundle

Actor	Usuario registrado
Requisitos	RF-14, RF-15 y RF-30
Precondición	El usuario ha iniciado sesión y existe una selección válida.
Flujo principal	1. Introduce nombre y una descripción. 2. Revisa las aplicaciones. 3. Guarda el bundle como privado. 4. Opcionalmente cambia la visibilidad a pública. 5. El sistema genera un enlace compartible.
Excepciones	Nombre inválido, aplicaciones eliminadas o intento de marcarlo como oficial sin permisos.
Postcondición	El bundle queda asociado a su propietario y conserva una composición persistente.

Tabla B.4: CU-02 Crear y publicar bundle.

CU-03 Personalizar un bundle público

Actor	Cualquier usuario
Requisitos	RF-16 y RF-17
Precondición	El bundle es público o pertenece al usuario.
Flujo principal	1. El usuario abre el bundle. 2. Añade o retira aplicaciones para esa descarga. 3. Genera el ZIP con la composición temporal.
Postcondición	El bundle original no cambia. Un usuario registrado puede elegir clonar la composición como uno nuevo.

Tabla B.5: CU-03 Personalizar un bundle público.

CU-04 Asignar estrella

Actor	Usuario registrado
Requisitos	RF-18 y RF-19
Precondición	El bundle es público y el usuario puede valorarlo.
Flujo principal	El usuario activa o retira la estrella. El backend crea o elimina la relación única y actualiza el contador.
Excepciones	Bundle privado, inexistente o restricción de valorar el propio bundle.
Postcondición	El ranking refleja el nuevo número de estrellas.

Tabla B.6: CU-04 Asignar estrella.

CU-05 Mantener una fuente

Actor	Administrador
Requisitos	RF-21 a RF-25
Precondición	El administrador está autenticado.
Flujo principal	1. Selecciona una aplicación. 2. Configura sistema, arquitectura, página oficial, dominios y resolver. 3. Ejecuta una prueba. 4. Revisa URL final, redirecciones y metadatos. 5. Guarda la fuente si el resultado es válido.
Excepciones	Dominio no permitido, red privada, contenido inesperado o resolución fallida.
Postcondición	La configuración queda versionada y la acción se registra.

Tabla B.7: CU-05 Mantener una fuente.

CU-05b Solicitar una aplicación

Actor	Usuario registrado
Requisitos	RF-20
Precondición	El usuario ha iniciado sesión.
Flujo principal	1. Introduce nombre, descripción redactada manualmente y página oficial de la aplicación propuesta. 2. Envía la solicitud. 3. El sistema la registra con estado pendiente de revisión.
Excepciones	Datos obligatorios ausentes, URL inválida o solicitud duplicada para la misma aplicación.
Postcondición	La propuesta queda disponible para revisión administrativa sin incorporarse todavía al catálogo público.

Tabla B.8: CU-05b Solicitar una aplicación.

CU-06 Buscar mediante lenguaje natural

Actor	Cualquier usuario
Requisitos	RF-26 a RF-28; RNF-14
Precondición	La búsqueda semántica está habilitada.
Flujo principal	1. El usuario escribe una necesidad. 2. El sistema genera un embedding y recupera candidatos. 3. Aplica filtros de estado y compatibilidad. 4. Opcionalmente solicita una explicación al LLM. 5. Valida y muestra los resultados.
Alternativas	Si falla el LLM, se muestran los candidatos sin explicación. Si no hay candidatos, se informa sin inventar aplicaciones.
Postcondición	El usuario puede añadir resultados a su selección o generar un borrador de bundle.

Tabla B.9: CU-06 Buscar mediante lenguaje natural.

B.8. Trazabilidad inicial

Área	Requisitos	Pruebas previstas
Catálogo y selección	RF-01 a RF-05	API, filtros, compatibilidad y pruebas E2E.
Descarga y ZIP	RF-06 a RF-12	Integración, descargas simuladas, ZIP parcial, cancelación y limpieza.
Usuarios y bundles	RF-13 a RF-19, RF-30	Autorización, propiedad, enlace, clonación y estrella única.
Administración	RF-20 a RF-25	Permisos, auditoría, resolver controlado y estados.
Semántica	RF-26 a RF-28	Recuperación, filtros, esquema inválido y caída del proveedor.
Calidad transversal	RNF-01 a RNF-18	Seguridad, carga, accesibilidad, observabilidad y privacidad.

Tabla B.10: Trazabilidad entre áreas y pruebas.

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo traduce los requisitos a una arquitectura implementable. Los nombres exactos de clases y tablas podrán adaptarse a las convenciones del código, pero deberán conservarse las responsabilidades, relaciones y restricciones descritas.

C.2. Diseño de datos

MySQL como fuente transaccional

Tabla C.1: Entidades transaccionales principales.

Entidad	Contenido y restricciones
<i>users</i>	Identidad, correo normalizado y verificado, nombre, rol, estado y marcas temporales. El correo será único.
<i>software_apps</i>	Nombre, slug único, descripción redactada manualmente por un usuario con permisos de administración, página oficial, icono, estado, creador y marcas temporales.
<i>tags</i> y <i>software_app_tags</i>	Etiquetas únicas y relación muchos a muchos con aplicaciones.

Entidad	Contenido y restricciones
<i>download_sources</i>	Aplicación, sistema operativo, arquitectura, región, país, URL de partida, tipo y configuración de resolver, dominios permitidos y estado.
<i>resolved_sources</i>	Resultado temporal de resolución: URL protegida, dominio final, nombre, tipo, tamaño, fecha de comprobación y expiración.
<i>bundles</i>	Propietario opcional para oficiales, nombre, slug, descripción redactada manualmente por su autor o por un administrador, visibilidad, marca oficial, estado, contador de estrellas y marcas temporales.
<i>bundle_items</i>	Relación única entre bundle y aplicación, con preferencias opcionales de sistema y arquitectura.
<i>bundle_stars</i>	Relación única usuario-bundle que representa una estrella activa.
<i>download_jobs</i>	Usuario o sesión anónima, estado, contexto, tamaños, progreso, referencia temporal al ZIP, errores y expiración.
<i>download_job_items</i>	Aplicación, fuente resuelta, nombre, tamaños, estado, error, reintentos y progreso individual.
<i>software_requests</i>	Solicitante, nombre, descripción propuesta manualmente por el usuario registrado, URL oficial propuesta, estado, respuesta administrativa y fechas.
<i>resolver_logs</i>	Fuente, fase, estado, mensaje, metadatos seguros y fecha.
<i>admin_action_logs</i>	Administrador, acción, tipo e identificador del objetivo, resumen del cambio y fecha.
<i>api_keys</i>	Usuario, prefijo visible, hash del secreto, estado, última utilización y fecha de revocación.

Los datos de configuración que puedan contener secretos no se almacenarán directamente en las tablas. Se utilizarán referencias a secretos externos. Las URL temporales podrán cifrarse o limitarse al almacenamiento del trabajo y nunca se enviarán a registros generales.

Estados

Las aplicaciones utilizarán los estados *active*, *pending review*, *broken*, *disabled*, *deprecated* y *requires manual review*. Solo *active* aparecerá como descargable.

Los trabajos utilizarán *queued*, *resolving*, *checking sizes*, *downloading*, *compressing*, *ready*, *failed*, *cancelled* y *expired*. Las transiciones válidas se comprobarán en un servicio de dominio; no se admitirán saltos arbitrarios desde controladores o workers.

Restricciones e índices

- Índices únicos para slugs, correos, etiqueta normalizada, estrella usuario-bundle y elemento bundle-aplicación.
- Índices compuestos para fuentes por aplicación, sistema, arquitectura, región y estado.
- Índices para trabajos por propietario, estado, creación y expiración.
- Índices para bundles públicos por estrellas, fecha y slug.
- Integridad referencial con eliminación restringida cuando una aplicación participe en trabajos o registros históricos.
- Borrado lógico para aplicaciones y bundles cuando sea necesario conservar auditoría.

PostgreSQL y pgvector

PostgreSQL contendrá una tabla derivada similar a:

```
software_embeddings
- software_app_id
- content_version
- model_id
- embedding
- generated_at
```

La clave lógica combinará aplicación, versión del contenido y modelo. Cuando cambien los metadatos relevantes se publicará un trabajo de regeneración. Los resultados semánticos devolverán identificadores; la API

consultará MySQL para obtener los datos vigentes y aplicar permisos y filtros.

C.3. Diseño arquitectónico

Capas del backend

API: controladores REST, autenticación, validación de DTO y documentación OpenAPI.

Aplicación: casos de uso, transacciones, autorización por propietario y coordinación.

Dominio: entidades, reglas, estados, políticas de bundles y objetos de valor.

Infraestructura: JPA, RabbitMQ, MinIO, Playwright, correo, embeddings y LLM.

Las dependencias apuntarán hacia las capas interiores. Los adaptadores externos implementarán interfaces definidas por la aplicación o el dominio, especialmente para resolución, almacenamiento temporal, colas y proveedores de IA.

Servicios desplegados

Componente	Responsabilidad	Escalado
Frontend	Interfaz estática y consumo de API	Réplicas o CDN
API Spring	Usuarios, catálogo, bundles, trabajos y administración	Por tráfico HTTP
Worker de resolución	Playwright y validación de fuentes	Por cola de resolución
Worker de descarga	Transferencia, progreso y reintentos	Por red y cola
Worker de ZIP	Compresión, manifiesto y publicación temporal	Por CPU, disco y cola
Servicio semántico	Embeddings, pgvector y coordinación del LLM	Por latencia y cuota
Scheduler	Comprobaciones y limpieza periódica	Una instancia con elección de líder

Tabla C.2: Componentes lógicos y criterio de escalado.

La primera implementación podrá combinar los workers en un único proceso. Los mensajes y puertos se diseñarán de forma que su separación posterior no cambie la API pública.

TODO: Contratos HTTP principales

Tabla C.3: Operaciones principales de la API.

Método	Ruta conceptual	Finalidad
GET	<i>/api/apps</i>	Buscar y filtrar el catálogo.
GET	<i>/api/apps/{slug}</i>	Consultar detalle y fuentes visibles.
POST	<i>/api/download-jobs</i>	Validar selección y crear trabajo.
GET	<i>/api/download-jobs/{id}</i>	Consultar estado y progreso.
DELETE	<i>/api/download-jobs/{id}</i>	Cancelar un trabajo.
GET	<i>/api/download-jobs/{id}/file</i>	Descargar un ZIP listo.

Método	Ruta conceptual	Finalidad
GET/POST	<i>/api/bundles</i>	Buscar o crear bundles.
GET/PATCH/DELETE	<i>/api/bundles/{slug}</i>	Consultar o mantener un bundle propio.
POST	<i>/api/bundles/{id}/star</i>	Asignar estrella.
DELETE	<i>/api/bundles/{id}/star</i>	Retirar estrella.
POST	<i>/api/software-requests</i>	Solicitar una aplicación.
POST	<i>/api/semantic-search</i>	Buscar por intención.
POST	<i>/api/bundle-suggestions</i>	Crear un borrador de bundle.
POST	<i>/api/admin/sources/{id}/test</i>	Probar una fuente y su resolver.

Los cuerpos y respuestas se definirán en OpenAPI. Los errores compartirán un esquema con código estable, mensaje público, identificador de incidencia y detalles de validación permitidos.

C.4. Diseño procedimental

Creación de un trabajo

1. Autenticar o identificar la sesión anónima.
2. Validar lista, sistema, arquitectura, región y formato.
3. Eliminar duplicados y comprobar que las aplicaciones están activas.
4. Aplicar cuotas y límites conocidos.
5. Crear `download_jobs` y sus elementos en una transacción.
6. Publicar el identificador en RabbitMQ mediante patrón de salida transaccional o mecanismo equivalente.
7. Responder con código de aceptación e identificador del trabajo.

Resolución segura

1. Seleccionar la fuente compatible más específica.
2. Comprobar si existe una resolución no expirada.

3. Ejecutar resolver dedicado o Playwright.
4. Validar HTTPS, hostname, DNS, IP y puerto.
5. Seguir un número limitado de redirecciones repitiendo la validación.
6. Comprobar dominio final, tipo de contenido, nombre y tamaño.
7. Persistir resultado y metadatos sin exponer secretos.
8. Marcar para revisión manual cuando el fallo sea permanente.

Descarga y reintentos

Cada elemento se descargará con un máximo configurable de intentos. Los fallos temporales utilizarán espera incremental. Si la fuente acepta rangos, una implementación posterior podrá reanudar; en la primera versión se reiniciará el archivo para simplificar la validación.

Durante la transferencia se controlará el número real de bytes. Al terminar se comparará con el tamaño anunciado cuando exista, se verificará el tipo y se rechazará cualquier archivo que exceda límites o cambie a un dominio no autorizado.

Compresión y finalización

1. Reunir elementos válidos y errores finales.
2. Sanitizar y resolver nombres duplicados.
3. Generar `manifest.json` y `README.txt`.
4. Construir el ZIP en el espacio temporal.
5. Publicarlo en MinIO con expiración.
6. Marcar el trabajo como listo o fallido.
7. Eliminar instaladores intermedios.
8. Enviar aviso si fue solicitado.

Cancelación y limpieza

La cancelación fijará una marca persistente y publicará una señal para los workers. Cada fase comprobará esa marca entre operaciones y cerrará recursos. El proceso eliminará objetos parciales y finalizará como *cancelled*. El scheduler buscará trabajos expirados o bloqueados y ejecutará una limpieza idempotente.

Búsqueda semántica

1. Validar longitud y filtros de la consulta.
2. Generar embedding mediante el proveedor configurado.
3. Recuperar los vecinos más próximos en pgvector.
4. Consultar las aplicaciones actuales en MySQL.
5. Aplicar estado, compatibilidad, seguridad y permisos.
6. Devolver resultados vectoriales.
7. Si se solicita explicación, enviar solo esos resultados al LLM.
8. Validar el esquema y descartar referencias no presentes.

Actualización del contador de estrellas

La estrella se insertará con una restricción única. El contador del bundle se actualizará en la misma transacción o mediante un evento idempotente. Una tarea administrativa podrá recalcularlo desde `bundle_stars` para reparar discrepancias.

C.5. Diseño de seguridad

Los secretos se proporcionarán mediante el entorno o un gestor de secretos. Los contenedores se ejecutarán sin privilegios, con sistemas de archivos de solo lectura cuando sea posible y límites de CPU, memoria y almacenamiento efímero.

Los logs utilizarán campos estructurados y excluirán contraseñas, claves, cabeceras de autorización, URL firmadas y contenido de instaladores. La auditoría registrará cambios de seguridad sin copiar secretos.

La salida del LLM se considerará entrada no confiable: no se renderizará como HTML sin sanitización, no se ejecutará como código y no se convertirá directamente en comandos o cambios de configuración.

C.6. Diseño de despliegue

El entorno local utilizará Docker Compose. El despliegue escalable utilizará Docker con:

- *Deployments* separados para API y workers;
- *Services* internos para componentes sin exposición pública;
- *Ingress* únicamente para frontend y API;
- comprobaciones de vida y disponibilidad;
- límites y solicitudes de recursos;
- volúmenes o almacenamiento de objetos para temporales controlados;
- secretos fuera de las imágenes;
- políticas de red que restrinjan el acceso entre componentes.

El escalado automático de workers se basará preferentemente en la longitud de las colas y en métricas de recursos. No se escalará Playwright sin límites, porque cada navegador consume memoria y conexiones externas.

Apéndice D

TODO: Documentación técnica de programación

D.1. Introducción

Este manual se completará cuando exista una implementación estable. Documentarlo antes obligaría a describir comandos, módulos y dependencias que todavía pueden cambiar. El anexo conserva la estructura que deberá rellenarse durante la fase de entrega.

D.2. Estructura de directorios

Pendiente de reflejar la estructura real de los repositorios del frontend, backend, workers, infraestructura, pruebas y documentación.

D.3. Configuración del entorno

Se documentarán versiones compatibles de Java, Maven, Node.js, Docker y herramientas auxiliares, así como las variables necesarias. Los secretos no se incluirán en el repositorio; se proporcionarán ejemplos sin valores reales.

D.4. Manual del programador

Se describirán arquitectura, convenciones, módulos, contratos internos, migraciones, resolvers, colas, almacenamiento temporal y proceso para añadir una aplicación o un proveedor.

D.5. Compilación, instalación y ejecución del proyecto

Se incluirán procedimientos reproducibles para desarrollo local, pruebas, construcción de imágenes y despliegue. Los comandos se validarán desde una copia limpia del repositorio.

D.6. Pruebas del sistema

El manual final indicará cómo ejecutar pruebas unitarias, integración, extremo a extremo, seguridad y carga; qué servicios de prueba se requieren; y cómo interpretar los informes.

D.7. Operación y resolución de incidencias

Se documentarán métricas, logs, colas, limpieza de temporales, rotación de secretos, copias de seguridad, fuentes rotas y recuperación de trabajos. Esta sección se escribirá a partir del comportamiento observado, no de supuestos.

Apéndice E

TODO: Documentación de usuario

E.1. Introducción

El manual de usuario definitivo se redactará cuando la interfaz esté implementada y validada. Se mantiene este esqueleto para asegurar que la entrega incluya instrucciones coherentes con las pantallas reales.

E.2. Requisitos de usuario

Se indicarán los navegadores soportados, las condiciones de conexión, los límites de descarga y los requisitos para cuentas, correo y funciones administrativas.

E.3. Acceso e instalación

Al tratarse de una aplicación web, el usuario normal no necesitará instalar Batch Downloader. Esta sección explicará cómo acceder al servicio y, si se distribuye una versión local, cómo iniciarla mediante contenedores.

E.4. Uso del catálogo

Se documentarán búsqueda, filtros, selección, tamaño estimado, revisión de fuentes, creación del trabajo, progreso, cancelación y descarga del ZIP.

E.5. Bundles y estrellas

Se explicará la diferencia entre personalizar temporalmente, clonar y editar un bundle propio; la visibilidad pública o privada; el enlace compatible; la estrella binaria; y que la descripción de cada bundle la redacta manualmente su autor o un administrador.

E.6. Búsqueda semántica

Se incluirán ejemplos de consultas por intención, explicación de resultados y advertencias sobre el carácter asistido de las recomendaciones.

E.7. Administración

El manual describirá alta y edición de aplicaciones, configuración de fuentes; prueba de resolvers; revisión de solicitudes; logs; y estados del catálogo.

E.8. Errores frecuentes y soporte

Se añadirán mensajes reales, causas, pasos de recuperación y forma de comunicar una incidencia mediante su identificador. No se incorporarán capturas ni instrucciones hasta que las pantallas sean definitivas.

Anexo de sostenibilización curricular

F.1. Introducción

Las directrices de la CRUE entienden la sostenibilidad como la búsqueda conjunta de calidad ambiental, justicia social y una economía viable y equitativa, y proponen cuatro competencias transversales: contextualización crítica, uso sostenible de recursos, participación y aplicación de principios éticos [1]. El diseño de Batch Downloader permite reflexionar sobre estas dimensiones antes de implementar el sistema y obliga a considerar consecuencias que no aparecen si el proyecto se analiza solo como un problema técnico.

F.2. SOS1: contextualización crítica

La utilidad de la plataforma nace de una práctica extendida: al preparar un ordenador, cada usuario repite búsquedas y descargas en múltiples páginas. Centralizar el catálogo puede ahorrar tiempo y reducir el acceso accidental a sitios no oficiales. Sin embargo, esa comodidad también concentra responsabilidad. El servicio procesa ejecutables de terceros, depende de proveedores globales y puede generar un consumo elevado de red, disco y energía si se usa de forma indiscriminada.

El análisis debe relacionar, por tanto, la experiencia individual con efectos más amplios. Una descarga fallida no es solo un error de programación: puede deberse a restricciones regionales, condiciones de uso, accesibilidad insuficiente o cambios comerciales del proveedor. La búsqueda semántica

también tiene implicaciones sociales, porque un modelo podría favorecer aplicaciones populares y ocultar alternativas pequeñas o libres. Para evitarlo, las recomendaciones se limitarán al catálogo controlado, mostrarán su fundamento y nunca sustituirán los filtros explícitos del usuario.

F.3. SOS2: uso sostenible de recursos

El principio técnico más directamente relacionado con la sostenibilidad es no almacenar instaladores de forma permanente. Mantener copias de cada versión multiplicaría el uso de almacenamiento y obligaría a gestionar duplicados obsoletos. Batch Downloader descargará los archivos únicamente durante el trabajo, generará el ZIP y eliminará los temporales al finalizar o expirar.

Esta decisión no elimina el impacto. Agrupar aplicaciones puede aumentar la transferencia si el usuario selecciona programas que después no necesita. La interfaz deberá mostrar tamaño estimado, fuentes y contenido antes de confirmar. Se aplicarán límites de tamaño, concurrencia y duración, y se reutilizarán metadatos de resolución durante periodos breves para evitar navegaciones repetidas. Los workers se escalarán según la carga real y los contenedores tendrán límites de recursos.

La inteligencia artificial merece una atención específica. Generar embeddings y consultar un LLM consume recursos adicionales. La búsqueda convencional se utilizará primero para nombres y filtros, reservando la semántica para consultas por intención. Las respuestas podrán servirse sin LLM cuando la búsqueda vectorial sea suficiente. En una fase posterior se compararán proveedor remoto y modelo local atendiendo no solo a calidad y coste, sino también a consumo energético.

F.4. SOS3: participación

Los bundles públicos, las estrellas y las solicitudes de software permiten que la comunidad contribuya al catálogo. Esta participación debe diseñarse para que no se convierta en spam, manipulación de rankings o difusión de fuentes dudosas. Solo los administradores podrán aprobar aplicaciones y dominios. Las estrellas serán binarias y trazables, mientras que los usuarios podrán clonar o personalizar un bundle sin alterar el trabajo de su autor.

La participación también implica escuchar problemas de accesibilidad y uso. El proyecto deberá ofrecer navegación por teclado, mensajes comprensi-

bles y mecanismos para informar de enlaces rotos. Los registros técnicos no bastan para conocer la experiencia: las pruebas con usuarios y la revisión de solicitudes deberán influir en la priorización.

F.5. SOS4: principios éticos

La plataforma deberá actuar con transparencia. Antes de descargar se explicará que los archivos proceden de fuentes oficiales, se procesan temporalmente y no se ejecutan. El manifiesto identificará aplicaciones y dominios, y los errores no se ocultarán detrás de mensajes genéricos.

La privacidad se aplicará mediante minimización de datos, plazos de conservación y protección de cuentas, direcciones IP, registros y correos. Los logs y el contexto enviado al LLM excluirán secretos y datos innecesarios. El modelo no podrá aprobar enlaces, cambiar dominios ni realizar acciones administrativas; sus sugerencias deberán ser revisables y validadas.

También existe una responsabilidad económica y social. La funcionalidad básica para usuarios anónimos evita convertir el acceso en una obligación de registro. Si se despliega públicamente, las cuotas deberán proteger la infraestructura sin discriminar a quienes tengan conexiones lentas. El proyecto respetará las licencias y condiciones de los editores, no presentará software de terceros como propio y ofrecerá información suficiente para que el usuario tome una decisión.

F.6. Conclusión personal

TODO: Esto es una locura.

Bibliografía

- [1] Conferencia de Rectores de las Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículum. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. Documento presentado a la Asamblea General de la CRUE el 28 de junio de 2012.
- [2] Jefatura del Estado. Real decreto legislativo 1/1996 por el que se aprueba el texto refundido de la ley de propiedad intelectual. <https://www.boe.es/eli/es/rdlg/1996/04/12/1>, 1996. Boletín Oficial del Estado.
- [3] Jefatura del Estado. Ley 34/2002, de servicios de la sociedad de la información y de comercio electrónico. <https://www.boe.es/eli/es/l/2002/07/11/34>, 2002. Boletín Oficial del Estado.
- [4] Jefatura del Estado. Ley orgánica 3/2018, de protección de datos personales y garantía de los derechos digitales. <https://www.boe.es/eli/es/lo/2018/12/05/3>, 2018. Boletín Oficial del Estado.
- [5] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (ue) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016. Consultado el 18 de junio de 2026.